CENTRO UNIVERSITÁRIO UNISENAI

CURSO DE GRADUAÇÃO TECNOLÓGICA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Trabalho de Teste de Carga e Stress

Qualidade e Teste de Software

Engenharia de Performance em E-sports

Aluno: João Victor Bagatim

FLORIANÓPOLIS

11/2026

DETALHAMENTO DO PROJETO

Título do projeto:	Teste de Carga e Stress em E-sports
Aluno:	João Victor Bagatim
Turma:	GR SADS 2024/2 N1
Curso:	Tecnologia em Análise e Desenvolvimento de Sistemas
Professor:	MSc. Hugo Menezes Barra

LINK GITHUB:

https://github.com/joaobagatim/SENAI/tree/main/QA/Trabalho-TesteCargaStress

1. Introdução

Este trabalho aplica os conceitos de teste de carga, teste de estresse e requisitos não funcionais apresentados na aula de Qualidade e Teste de Software. O cenário teórico considera a API da RiotForge, responsável pelas inscrições, estatísticas dos jogadores e pelo placar ao vivo de um campeonato de e-sports.

Na parte prática, foi utilizado o Locust para consultar a rota /users da API JSONPlaceholder. Foram executados o cenário principal de 50 usuários por três minutos e outros cinco testes, com cargas e durações diferentes, para comparar latência, vazão e taxa de erro. Os dados e as capturas apresentados correspondem às execuções realizadas em 6 de novembro de 2026.

2. Arquitetura e requisitos não funcionais sob pressão

2.1 Diferença entre teste de carga e teste de estresse

O RNF-PERF-01 deve ser avaliado com teste de carga, pois define uma capacidade esperada de operação.

A equipe de QA deve aumentar a carga gradualmente até atingir 10.000 requisições simultâneas.

Uma rampa possível é acrescentar 1.000 requisições concorrentes por etapa, mantendo cada patamar para medir o comportamento.

Ao chegar à carga prevista, a equipe deve sustentá-la e verificar se a latência permanece inferior a 800 ms.

Devem ser registrados os tempos de resposta, os percentis, o máximo, a vazão e as falhas em cada etapa.

Como o requisito não define um percentil, a média sozinha não comprova seu cumprimento; a interpretação deve ser alinhada com o PO.

Já o RNF-RES-02 exige teste de estresse, ultrapassando a capacidade normal em ambiente controlado.

Nesse teste, a rampa continua além do limite esperado até provocar, de forma monitorada, o esgotamento das conexões com o banco.

O QA verifica se a API rejeita o excesso de forma controlada, por exemplo com HTTP 503 e uma mensagem clara.

Depois, a equipe reduz a carga até o nível normal e inicia a medição do tempo de recuperação.

O fluxo deve voltar ao normal em menos de 60 segundos, sem reinício manual e sem deixar o serviço travado.

Portanto, o primeiro teste avalia desempenho na capacidade prevista; o segundo avalia falha controlada e recuperação após a sobrecarga.

3. Ponto de colapso e métricas

O ponto de colapso é a carga em que o sistema perde sua capacidade de atender de forma aceitável e sustentada. A conclusão deve combinar métricas ao longo do tempo; um pico isolado de latência não comprova colapso.

Latência: tempo entre o envio de uma requisição e sua resposta. O aumento persistente de p95/p99 e do tempo máximo indica filas e dificuldade de atendimento. P95 representa o tempo em que aproximadamente 95% das requisições terminam. Exceder 800 ms viola a meta de desempenho, mas não comprova sozinho o colapso.

Vazão: número de requisições atendidas por segundo. Quando a demanda aumenta, mas a vazão deixa de crescer ou cai, há sinal de saturação que deve ser analisado junto da latência e das falhas.

Taxa de erro: falhas divididas pelo total de requisições, multiplicadas por 100. O crescimento de timeouts e respostas HTTP 500/504 indica perda de atendimento. Uma resposta 503 controlada pode representar a proteção esperada pelo RNF de resiliência, mesmo sendo registrada como falha no cliente.

Recursos: CPU ou memória saturadas, conexões com o banco esgotadas e filas crescentes ajudam a localizar a causa. Essas métricas exigem acesso ao servidor e não estavam disponíveis no teste público.

Para validar a recuperação, deve-se reduzir a sobrecarga até o nível normal e medir se o serviço restabelece o fluxo em menos de 60 segundos, sem reinício manual. Esse teste exige ambiente controlado da própria equipe.

4. Automação com Python e Locust

O ambiente de execução utilizou Python 3.11.9, Locust 2.43.1 e Windows. Cada usuário virtual executou GET em /users e aguardou um intervalo aleatório entre 2 e 4 segundos antes da próxima consulta.

Arquivo: locustfile_esports.py

```
"""Teste de consulta ao perfil dos jogadores, conforme o enunciado."""

from locust import HttpUser, between, task


class Jogador(HttpUser):
    host = "https://jsonplaceholder.typicode.com"
    wait_time = between(2, 4)

    @task
    def consultar_jogadores(self):
        self.client.get("/users")
```

Comandos para reprodução:

```
python -m pip install -r requirements.txt
python -m locust -f locustfile_esports.py
```

No painel http://localhost:8089, o teste principal utiliza 50 usuários, entrada de 5 por segundo, host https://jsonplaceholder.typicode.com e duração de 3 minutos. A rampa de aproximadamente 10 segundos faz parte desse período.

As execuções foram iniciadas pela interface HTTP do próprio painel, acompanhadas na aba Charts e encerradas pelo tempo configurado. Os seis cenários foram executados em sequência, com estatísticas reiniciadas entre eles.

Usuários virtuais não equivalem a requisições simultâneas, pois parte deles fica em espera entre consultas. Esta prática não simula as 10.000 requisições simultâneas do requisito teórico e não executa atualização de placar.

5. Resultados dos seis testes

Teste	Usuários	Entrada/s	Tempo (s)	Requisições	Erros (%)	Média (ms)	p95 (ms)	Req./s
T1	50	5	180	2879	0,00	60,98	160	16,02
T2	10	2	45	143	0,00	54,03	100	3,20
T3	25	5	60	481	0,42	72,34	200	8,01
T4	40	5	75	946	0,00	56,43	150	12,62
T5	60	5	90	1669	0,00	49,73	110	18,56
T6	75	5	120	2813	0,00	53,76	130	23,44

Os tempos incluem a rampa de entrada. A média, o p95 e a vazão representam o cenário completo. Os percentis são aproximados pelo agrupamento de tempos do Locust. Cada teste foi realizado uma vez; portanto, a comparação não isola variações da rede ou do serviço.

O cenário de 10 usuários registrou 3,20 requisições por segundo e média de 54,03 ms. Com 75 usuários, a vazão foi de 23,44 requisições por segundo e a média foi de 53,76 ms. A pausa entre consultas limita a vazão produzida por cada usuário.

No T3, foram registradas 2 falhas: RemoteDisconnected('Remote end closed connection without response'). A ocorrência foi observada no cliente. Sem logs internos, não é possível atribuir a causa ao servidor, à rede ou a um intermediário.

As estatísticas completas e as seis capturas estão disponíveis nas pastas resultados e evidencias do repositório. A próxima página apresenta o print solicitado do teste principal.

6. Evidência visual do teste de três minutos

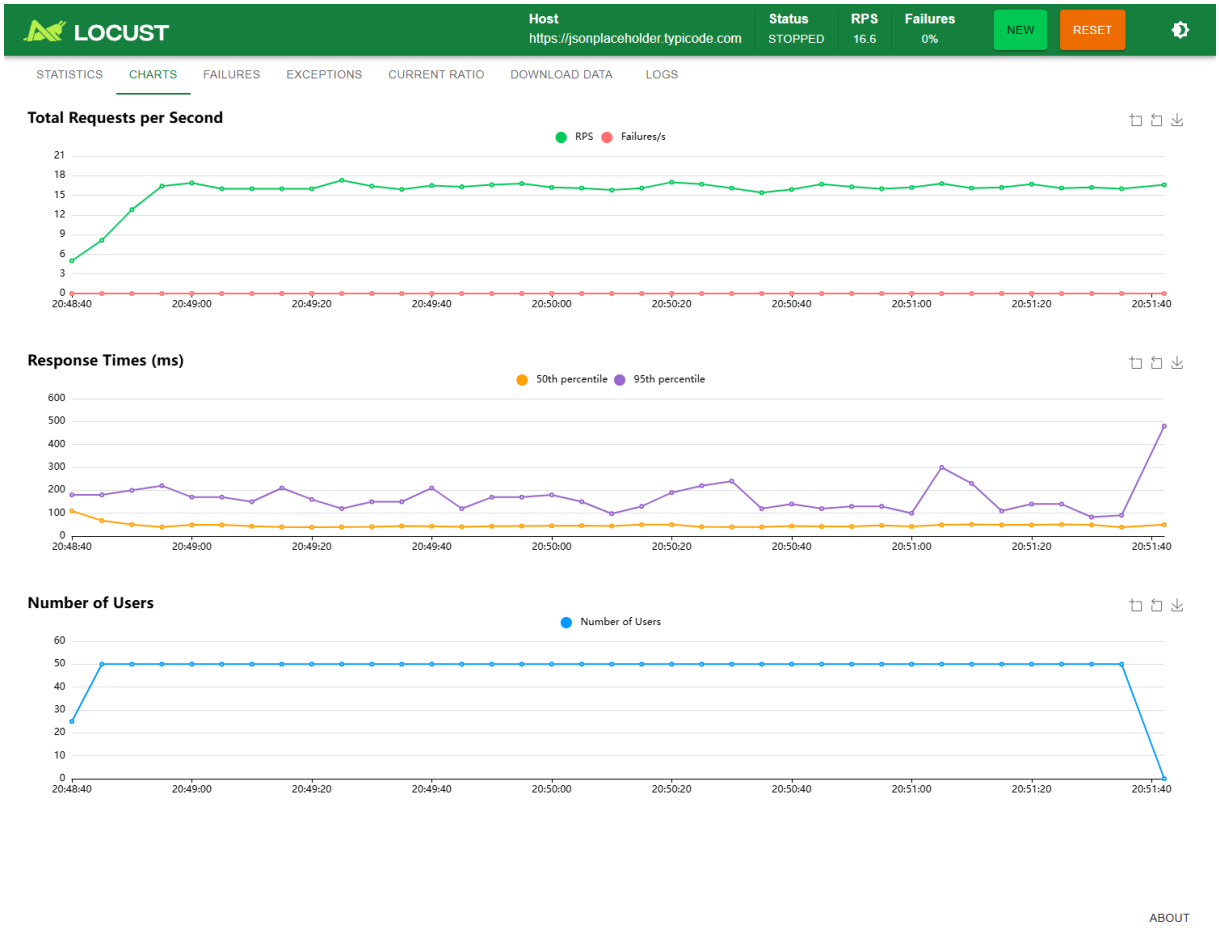


Figura 1 — Aba Charts após o T1: 50 usuários, entrada de 5 usuários/s e duração de 180 segundos. A redução de usuários ao final corresponde ao encerramento programado.

O teste principal registrou 2879 requisições, 0 falhas, latência média de 60,98 ms, p95 de 160 ms e máximo de 860,00 ms. A vazão média foi de 16,02 requisições por segundo.

A linha verde mostra requisições por segundo; a vermelha, falhas por segundo. Nos tempos de resposta, a linha laranja representa a mediana e a roxa o p95. O terceiro gráfico mostra a quantidade de usuários. As cores seguem a versão do Locust instalada.

7. Análise de engenharia e considerações finais

Nos seis cenários foram realizadas 8931 requisições e registradas 2 falhas pelo Locust.

No teste principal, a taxa de erro foi de 0,00% e a latência média ficou em 60,98 ms.

O p95 do teste principal foi de 160 ms, enquanto o máximo chegou a 860,00 ms.

Assim, o teste principal apresentou 0% de erro; isso demonstra sucesso das requisições segundo a classificação HTTP do Locust.

O maior p95 entre os seis cenários foi de 200 ms; o valor ajuda a avaliar a latência da maioria das respostas.

No teste principal, os dados indicam atendimento estável e desempenho favorável na consulta de perfis.

Para o T1, a API passaria em uma avaliação limitada de eficiência de desempenho que adotasse 0% de erro e p95 inferior a 800 ms.

Considerando todos os cenários, a API não manteve 0% de erro; as falhas registradas impedem afirmar estabilidade total.

Esse critério prático não substitui o RNF-PERF-01: o enunciado exige menos de 800 ms para até 10.000 requisições simultâneas.

Além de a carga ser menor, a prática usa GET /users, enquanto o requisito teórico trata da atualização do placar ao vivo.

Não houve acesso à CPU, à memória nem ao banco da API pública; a observação se limita ao comportamento visto pelo cliente.

Uma resposta rápida pode envolver cache, e variações da rede também afetam a latência medida; não foi identificada a arquitetura interna do fornecedor.

As métricas devem ser observadas em conjunto para distinguir oscilações isoladas de um ponto de colapso sustentado.

O RNF-RES-02 não foi validado, pois não houve indução controlada de esgotamento do banco seguida de medição da recuperação.

Uma avaliação completa da RiotForge exige ambiente próprio, carga compatível com o requisito e testes de recuperação após a sobrecarga.

REFERÊNCIAS

BARRA, Hugo Menezes. Aula 11 – Teste de Stress e Requisitos Não Funcionais. Material da disciplina Qualidade e Teste de Software fornecido para esta atividade.

BARRA, Hugo Menezes. Trabalho – Teste de Stress: Engenharia de Performance e Teste de Estresse em E-sports. Enunciado da atividade fornecido pelo professor.

LOCUST. Writing a locustfile. Disponível em: https://docs.locust.io/en/stable/writing-a-locustfile.html. Acesso em: 6 nov. 2026.

LOCUST. Running without the web UI. Disponível em: https://docs.locust.io/en/stable/running-without-web-ui.html. Acesso em: 6 nov. 2026.